# Từ A, B đến kết quả: GEMM, systolic array và VPU

Công thức và các bước tính, kèm một ví dụ số tính tay từ đầu đến cuối

Bộ tăng tốc GEMM INT8 trên Chipyard SoC

**CÁCH ĐỌC**

Mục 1 cho bức tranh chung. Mục 2 và 3 giải thích bias và lượng tử hóa, là nền tảng để hiểu VPU. Mục 4, 5, 6 lần lượt là GEMM, systolic array và VPU. Mục 7 ghép tất cả thành một thuật toán đầy đủ. Mục 8 là các điểm tinh chỉnh yêu cầu rút ra từ tài liệu này. Một ví dụ số duy nhất (M = 2, K = 4, N = 4, P = 2) được dùng xuyên suốt, nên có thể kiểm tra lại từng con số bằng tay.

# Mục lục

**1 Bức tranh tổng thể** **3**
1.1 Một nơ-ron . . . 3
1.2 Một lớp, nhiều mẫu cùng lúc . . . 3

**2 Bias là gì** **4**
2.1 Trực giác . . . 4
2.2 Bias cộng vào đâu . . . 4

**3 Lượng tử hóa INT8** **5**
3.1 Biểu diễn đối xứng . . . 5
3.2 C có scale gì . . . 5
3.3 Bias INT32 . . . 5
3.4 Requantize: đưa về scale của đầu ra . . . 5
3.5 Ví dụ số . . . 6

**4 GEMM** **7**
4.1 Ví dụ số . . . 7
4.2 Vì sao N, K là bội của 4 . . . 7

**5 Systolic array Output-Stationary** **8**
5.1 Chia tile . . . 8
5.2 Làm lệch dữ liệu (skew) . . . 8
5.3 Mỗi PE làm gì trong một chu kỳ . . . 9
5.4 Xả kết quả theo hàng . . . 9
5.5 Tile kế tiếp và tile biên . . . 9

**6 VPU: hậu xử lý** **10**
6.1 Vì sao bước 3 làm tròn đúng . . . 10
6.2 Vì sao cần 64 bit và không bao giờ tràn . . . 10
6.3 Ví dụ số . . . 10

**7 Toàn bộ thuật toán** **11**
7.1 Chuẩn bị trên PC (một lần cho mỗi mô hình) . . . 11
7.2 Trên accelerator (mỗi lần gọi) . . . 11
7.3 Khi lớp lớn hơn 32 . . . 11

**8 Tinh chỉnh yêu cầu** **12**

# 1 Bức tranh tổng thể

Một lớp của mạng nơ-ron tính **Y** = ReLU(**XW**+**b**). Phần nặng nhất là phép nhân ma trận **XW**; đó là việc của **systolic array**. Phần còn lại (cộng bias, đưa kết quả về INT8, ReLU) là việc của **VPU**. Kết quả Y lại là đầu vào A của lớp kế tiếp.

## 1.1 Một nơ-ron

Một nơ-ron nhận K giá trị đầu vào $x_0, \ldots, x_{K-1}$, nhân mỗi giá trị với một trọng số, cộng lại, cộng thêm một hằng số riêng của nó là **bias** $b$, rồi cho qua hàm kích hoạt:

$$y = \mathrm{ReLU}\Big(\sum_{k=0}^{K-1} x_k\, w_k + b\Big), \qquad \mathrm{ReLU}(z) = \max(z, 0).$$

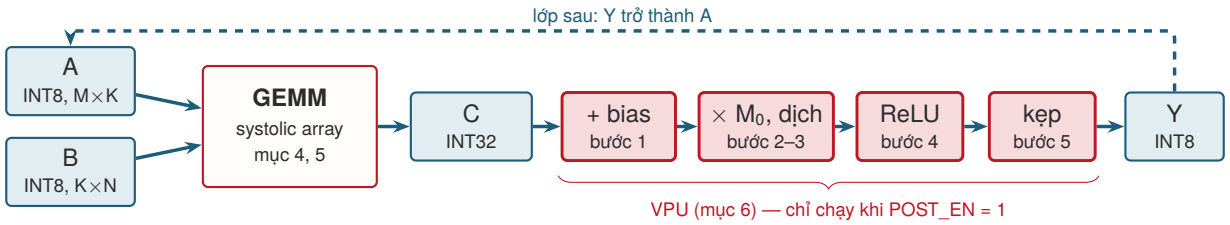
## 1.2 Một lớp, nhiều mẫu cùng lúc

Một lớp có N nơ-ron; mỗi lần ta đưa vào M mẫu (ví dụ M ảnh). Viết gọn dưới dạng ma trận:

$$Y[i][j] = \mathrm{ReLU}\Big(\underbrace{\sum_{k=0}^{K-1} X[i][k]\, W[k][j]}_{\text{GEMM: } C[i][j]} + b[j]\Big), \qquad 0 \le i < M,\ 0 \le j < N.$$

**Bảng 1.1:** Ánh xạ lớp nơ-ron sang tên gọi trong spec

| Tên toán học | Tên trong spec | Ý nghĩa |
|---|---|---|
| X (M×K) | A | Activation: mỗi hàng là một mẫu, mỗi cột là một đặc trưng đầu vào |
| W (K×N) | B | Trọng số: mỗi cột là một nơ-ron, gồm K trọng số của nó |
| XW (M×N) | C | Tổng tích lũy INT32, chưa có bias |
| b (N) | bias | Một hằng số cho mỗi nơ-ron, tức mỗi cột j |
| Y (M×N) | Y | Đầu ra INT8 của lớp; là A của lớp sau |




**Hình 1.1:** Đường đi của dữ liệu trong một lớp

# 2 Bias là gì

Bias là **một hằng số cộng thêm cho mỗi nơ-ron**, học được trong quá trình huấn luyện, giống hệ số $b$ trong đường thẳng $y = wx + b$. Mỗi cột j của C có một bias $b[j]$; mọi hàng (mọi mẫu) dùng chung bias đó.

## 2.1 Trực giác

- Không có bias, khi mọi đầu vào bằng 0 thì $\sum x_k w_k = 0$ và nơ-ron luôn ra 0. Bias cho phép nơ-ron có giá trị "mặc định" khác 0.
- Kết hợp với ReLU, bias đóng vai trò **ngưỡng**: nơ-ron chỉ "bật" khi $\sum x_k w_k > -b$. Bias dương làm nơ-ron dễ bật; bias âm làm nó khó bật.
- Bias là tham số của mô hình, nằm trong file trọng số cùng với W. Phần cứng không tính ra bias; nó chỉ đọc bias từ DRAM và cộng vào.

## 2.2 Bias cộng vào đâu

Bias cộng theo **cột**: cùng một $b[j]$ cho cả cột j của C.

$$\begin{bmatrix} C_{00} & C_{01} & C_{02} & C_{03} \\ C_{10} & C_{11} & C_{12} & C_{13} \end{bmatrix} + \begin{bmatrix} b_0 & b_1 & b_2 & b_3 \\ b_0 & b_1 & b_2 & b_3 \end{bmatrix}$$

Vì vậy bias là một vector N phần tử. VPU đọc $b[j_0], \ldots, b[j_0 + P - 1]$ cho P cột của tile đang xả.

**VÌ SAO BIAS PHẢI CỘNG TRƯỚC KHI REQUANTIZE, VÀ TRÊN INT32**
Bias thuộc về tổng $\sum x_k w_k + b$, nên phải cộng vào C khi C còn đủ độ chính xác (INT32), trước khi thu nhỏ về INT8. Nếu cộng sau khi đã làm tròn về INT8, sai số làm tròn của C và của bias cộng dồn vào nhau. Đây đúng là thứ tự mà spec quy định (FUN-03).

# 3 Lượng tử hóa INT8

Mạng được huấn luyện bằng số thực. Để chạy trên phần cứng số nguyên, mỗi tensor số thực $r$ được biểu diễn bằng số nguyên $q$ nhân với một **scale** $s$: $r \approx s \cdot q$. Phép nhân ma trận làm trên $q$; các scale được gom lại thành một hệ số duy nhất $M$, và VPU nhân với $M$ để đưa kết quả về INT8.

## 3.1 Biểu diễn đối xứng

$$r \approx s \cdot q, \qquad q = \text{clamp}\Big(\text{round}\big(\tfrac{r}{s}\big), -127, 127\Big), \qquad s = \frac{\max|r|}{127}.$$

"Đối xứng" nghĩa là số thực 0 ứng với số nguyên 0 (không có zero-point). Ta dùng ba scale: $s_x$ cho activation A, $s_w$ cho trọng số B, $s_y$ cho đầu ra Y. Các scale được tính một lần trên PC, khi chuẩn bị mô hình.

## 3.2 C có scale gì

$$\sum_k r_x[i][k]\, r_w[k][j] \approx \sum_k (s_x q_x)(s_w q_w) = s_x s_w \underbrace{\sum_k q_x[i][k]\, q_w[k][j]}_{C[i][j]}.$$

Vậy **C là số nguyên với scale** $s_x s_w$. Tổng này cần nhiều bit hơn 8: mỗi tích tới $2^{14}$, cộng 32 tích tới $2^{19}$, nên C dùng INT32.

## 3.3 Bias INT32

Muốn cộng bias trực tiếp vào C, bias phải có **cùng scale** với C:

$$b_q[j] = \text{round}\Big(\frac{b[j]}{s_x s_w}\Big) \quad \text{(INT32)}, \qquad acc[i][j] = C[i][j] + b_q[j] \;\approx\; \frac{\sum_k r_x r_w + b}{s_x s_w}.$$

Bias được lượng tử hóa trên PC và ghi vào DRAM như một vector INT32.

## 3.4 Requantize: đưa về scale của đầu ra

Ta cần $q_y$ sao cho $s_y q_y \approx s_x s_w \cdot acc$, tức

$$q_y = \text{round}(M \cdot acc), \qquad M = \frac{s_x s_w}{s_y}.$$

$M$ là số thực (thường nhỏ hơn 1). Phần cứng không dùng số thực, nên $M$ được viết dưới dạng **số nguyên nhân với lũy thừa của 2**:

$$M \approx M_0 \cdot 2^{-(31+sh)}, \qquad M_0 \in [2^{30}, 2^{31}), \quad sh \in \{0, \ldots, 31\}.$$

Cách tính trên PC (Python):

```
import math
m, e = math.frexp(M)          # M = m * 2**e, m trong [0.5, 1)
M0 = round(m * 2**31)         # M0 trong [2**30, 2**31]
sh = -e                       # cần sh >= 0, tức M < 1
if M0 == 2**31: M0 //= 2; sh -= 1
```

## 3.5 Ví dụ số

Dùng xuyên suốt tài liệu: $s_x = 0{,}05$, $s_w = 0{,}02$, $s_y = 0{,}04$.

$$M = \frac{0{,}05 \cdot 0{,}02}{0{,}04} = 0{,}025 = 0{,}8 \cdot 2^{-5} \;\Rightarrow\; M_0 = \text{round}(0{,}8 \cdot 2^{31}) = 1\,717\,986\,918, \quad sh = 5.$$

Kiểm tra: $1\,717\,986\,918 \cdot 2^{-36} \approx 0{,}0250000$. Bias số thực $b = (0{,}5;\ -0{,}3;\ 0{,}2;\ -1{,}0)$ chia cho $s_x s_w = 0{,}001$ thành $b_q = (500;\ -300;\ 200;\ -1000)$.

# 4 GEMM

GEMM (General Matrix Multiply) tính C = A × B. Mỗi phần tử $C[i][j]$ là tích vô hướng của hàng i của A với cột j của B. Tổng cộng có M·N·K phép nhân-cộng (MAC).

$$C[i][j] = \sum_{k=0}^{K-1} A[i][k] \cdot B[k][j], \qquad A, B \in \text{INT8},\ C \in \text{INT32}.$$

## 4.1 Ví dụ số

$$A = \begin{bmatrix} 12 & -5 & 30 & 7 \\ -20 & 8 & 4 & 15 \end{bmatrix}_{2\times 4} \qquad B = \begin{bmatrix} 3 & -2 & 10 & 5 \\ 6 & 4 & -7 & 1 \\ -4 & 9 & 2 & -3 \\ 8 & -1 & 5 & 6 \end{bmatrix}_{4\times 4}$$

$$C[0][0] = 12 \cdot 3 + (-5) \cdot 6 + 30 \cdot (-4) + 7 \cdot 8 = 36 - 30 - 120 + 56 = -58.$$

$$C = \begin{bmatrix} -58 & 219 & 250 & 7 \\ 92 & 93 & -173 & -14 \end{bmatrix}_{2\times 4}$$

## 4.2 Vì sao N, K là bội của 4

DRAM và DMA làm việc theo word 32 bit = 4 phần tử INT8. Khi N, K là bội của 4, mỗi hàng của A và B chiếm đúng một số nguyên word, nên không có word nào bị “dở dang”. Nếu ma trận thật không thỏa, phần mềm đệm thêm cột 0: phép nhân với 0 không làm thay đổi tổng.

# 5 Systolic array Output-Stationary

Array có P×P PE. Mỗi PE giữ **một** phần tử của C và cộng dồn nó tại chỗ (output-stationary). Ma trận C lớn được chia thành các tile P×P; array tính lần lượt từng tile. Trong một tile, A chảy từ trái sang, B chảy từ trên xuống, và mỗi PE nhận đúng cặp $A[i][k]$, $B[k][j]$ mà nó cần, nhờ dữ liệu được làm lệch thời gian.

## 5.1 Chia tile

$$T_M = \left\lceil \frac{M}{P} \right\rceil, \quad T_N = \left\lceil \frac{N}{P} \right\rceil, \qquad \text{tile } (t_i, t_j) \text{ phủ } i \in [i_0, i_0 + P),\ j \in [j_0, j_0 + P),\ i_0 = t_i P,\ j_0 = t_j P.$$

Thuật toán tương đương viết bằng vòng lặp (thứ tự tile tự do):

```
for ti in range(ceil(M/P)):
  for tj in range(ceil(N/P)):
    i0, j0 = ti*P, tj*P
    acc[r][c] = 0  (moi PE)            # thuc te: co first, khong can chu ky xoa
    for k in range(K):                 # ca K trong mot luot
      for r, c in PxP:                 # P*P PE chay song song
        acc[r][c] += A[i0+r][k] * B[k][j0+c]
    for r in range(P):                 # xa tung hang
      if i0+r < M:
        C[i0+r][j0 : j0+P] = acc[r][:]   (chi cot j0+c < N)
```

Ví dụ: M = 2, N = 4, P = 2 cho $T_M = 1$, $T_N = 2$: tile 0 là cột 0–1, tile 1 là cột 2–3. Với M = 20, N = 12, P = 8 có 3 × 2 = 6 tile; tile ở hàng 16–19 hoặc cột 8–11 là *tile biên*, có PE không dùng.

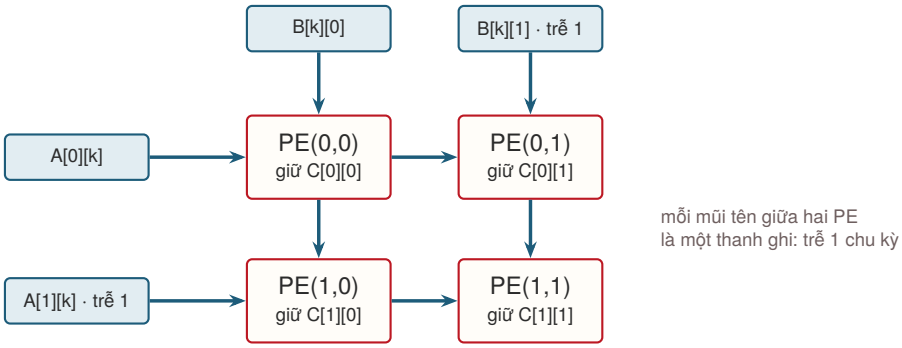
## 5.2 Làm lệch dữ liệu (skew)

Ở bước k, SRAM trả về một cột của khối A (P phần tử $A[i_0 + r][k]$) và một hàng của khối B (P phần tử $B[k][j_0 + c]$), cùng một chu kỳ. Array không đưa chúng vào thẳng mà làm trễ:

- lane r của A trễ r chu kỳ, rồi đi sang phải, mỗi PE thêm 1 chu kỳ;
- lane c của B trễ c chu kỳ, rồi đi xuống, mỗi PE thêm 1 chu kỳ.

Kết quả là PE(r, c) nhận $A[i_0 + r][k]$ và $B[k][j_0 + c]$ **cùng một chu kỳ**:

$$t_{r,c}(k) = t_0 + k + r + c.$$




**Hình 5.1:** Array 2×2 cho tile 0 của ví dụ (cột 0–1)

**Bảng 5.1:** PE nào nhân gì ở chu kỳ nào (tile 0, $t_0$ = 0, K = 4)

| Chu kỳ | PE(0,0) | PE(0,1) | PE(1,0) | PE(1,1) |
|---|---|---|---|---|
| 0 | $A_{00}B_{00}$ *first* | — | — | — |
| 1 | $A_{01}B_{10}$ | $A_{00}B_{01}$ *first* | $A_{10}B_{00}$ *first* | — |
| 2 | $A_{02}B_{20}$ | $A_{01}B_{11}$ | $A_{11}B_{10}$ | $A_{10}B_{01}$ *first* |
| 3 | $A_{03}B_{30}$ *last* | $A_{02}B_{21}$ | $A_{12}B_{20}$ | $A_{11}B_{11}$ |
| 4 | tile 1, k = 0 | $A_{03}B_{31}$ *last* | $A_{13}B_{30}$ *last* | $A_{12}B_{21}$ |
| 5 | tile 1, k = 1 | tile 1, k = 0 | tile 1, k = 0 | $A_{13}B_{31}$ *last* |

Tổng tích lũy của PE(0,0) qua các chu kỳ 0 … 3: $36 \to 6 \to -114 \to -58 = C[0][0]$.

## 5.3 Mỗi PE làm gì trong một chu kỳ

$$sum = \begin{cases} a \cdot b & \text{nếu } first \\ acc + a \cdot b & \text{ngược lại} \end{cases} \qquad acc \leftarrow sum, \qquad \text{nếu } last: \ res \leftarrow sum.$$

- Cờ **first** đi kèm phần tử k = 0: PE nạp tích mới thay vì cộng vào giá trị cũ, nên không cần chu kỳ xóa giữa hai tile.
- Cờ **last** đi kèm phần tử k = K−1: PE chép tổng cuối vào thanh ghi kết quả **res**. `acc` được giải phóng ngay để tính tile sau, trong khi `res` chờ được xả.
- a, cờ hợp lệ, first, last đi sang PE bên phải; b đi xuống PE bên dưới.

## 5.4 Xả kết quả theo hàng

PE(r, c) nhận phần tử cuối ở chu kỳ $t_0 + K - 1 + r + c$. Hàng r có đủ P kết quả khi cột cuối c = P−1 xong:

hàng $r$ sẵn sàng ở chu kỳ $t_0 + K + P - 1 + r$.

Các hàng sẵn sàng lần lượt, mỗi chu kỳ một hàng, nên array xả đúng **một hàng mỗi chu kỳ** sang VPU. Trong ví dụ, hàng 0 của tile 0 ($C_{00}, C_{01}$) sẵn sàng ở chu kỳ 5, hàng 1 ở chu kỳ 6.

## 5.5 Tile kế tiếp và tile biên

- Tile kế tiếp bắt đầu sau $\max(K, P)$ chu kỳ. Điều kiện này bảo đảm hàng r của tile trước đã được xả trước khi tile sau ghi đè `res` của hàng đó, và cổng ghi không bị hai hàng dùng cùng lúc. Trong ví dụ, tile 1 bắt đầu ở chu kỳ 4 (bảng 5.1).
- Thời gian tính xấp xỉ: $T_M T_N \cdot \max(K, P) + 2P$ chu kỳ.
- Tile biên: hàng r chỉ hợp lệ khi $i_0 + r < M$, cột c khi $j_0 + c < N$. PE không hợp lệ vẫn tính (trên dữ liệu rác), nhưng hàng không hợp lệ không được xả và cột không hợp lệ có mask = 0, nên không vị trí sai nào bị ghi.

# 6 VPU: hậu xử lý

VPU nhận từng hàng INT32 từ array, mỗi chu kỳ một hàng P phần tử, và áp dụng 5 bước cho từng phần tử. Bước 2–3 là cách làm phép nhân với số thực $M$ bằng số nguyên. Khi POST_EN = 0, VPU cho C đi thẳng, không đổi.

**Bảng 6.1:** Năm bước của VPU

| Bước | Công thức | Ý nghĩa |
|---|---|---|
| 1. Cộng bias | $acc = C[i][j] + b_q[j]$ (INT32) | Hoàn thành $\sum xw + b$ ở scale $s_x s_w$ |
| 2. Nhân | $prod = acc \cdot M_0$ (64 bit) | Phần "số nguyên" của phép nhân với $M$ |
| 3. Dịch có làm tròn | $y = (prod + 2^{30+sh}) \gg (31 + sh)$ | Phần "lũy thừa 2" của $M$. Cộng nửa đơn vị trước khi dịch để làm tròn thay vì cắt bỏ |
| 4. ReLU | $y = \max(y, 0)$ nếu RELU_EN | Hàm kích hoạt; lớp cuối thường tắt |
| 5. Kẹp | $Y = \min(\max(y, -128), 127)$ | Đưa về miền INT8 |

## 6.1 Vì sao bước 3 làm tròn đúng

Dịch phải $n$ bit là chia cho $2^n$ rồi lấy phần nguyên dưới (floor). Cộng thêm $2^{n-1}$ (nửa đơn vị) trước khi dịch biến floor thành làm tròn tới số gần nhất; trường hợp đúng nửa được làm tròn lên. Với $n = 31 + sh$ thì $2^{n-1} = 2^{30+sh}$.

## 6.2 Vì sao cần 64 bit và không bao giờ tràn

$|acc| < 2^{31}$ và $|M_0| < 2^{31}$ nên $|prod| < 2^{62}$; cộng thêm $2^{30+sh} \le 2^{61}$ vẫn nhỏ hơn $2^{63}$. Phép tính 64 bit có dấu luôn đủ chỗ.

## 6.3 Ví dụ số

Với C ở mục 4, $b_q = (500; -300; 200; -1000)$, $M_0 = 1\,717\,986\,918$, $sh = 5$ (dịch 36 bit), ReLU bật:

**Bảng 6.2:** VPU trên từng phần tử của ví dụ

| (i, j) | C | acc | prod | y | Y | số thực | $Y \cdot s_y$ |
|---|---|---|---|---|---|---|---|
| (0,0) | −58 | 442 | 759 350 217 756 | 11 | 11 | 0,442 | 0,44 |
| (0,1) | 219 | −81 | −139 156 940 358 | −2 | 0 | 0 | 0 |
| (0,2) | 250 | 450 | 773 094 113 100 | 11 | 11 | 0,450 | 0,44 |
| (0,3) | 7 | −993 | −1 705 961 009 574 | −25 | 0 | 0 | 0 |
| (1,0) | 92 | 592 | 1 017 048 255 456 | 15 | 15 | 0,592 | 0,60 |
| (1,1) | 93 | −207 | −355 623 292 026 | −5 | 0 | 0 | 0 |
| (1,2) | −173 | 27 | 46 385 646 786 | 1 | 1 | 0,027 | 0,04 |
| (1,3) | −14 | −1014 | −1 742 038 734 852 | −25 | 0 | 0 | 0 |

Cột "số thực" là ReLU($\sum r_x r_w + b$) tính bằng số thực; cột cuối là giá trị Y quy ngược về số thực. Hai cột gần bằng nhau: sai khác chỉ do làm tròn về INT8. Kiểm tra tay ô (0,0): $442 \cdot 1\,717\,986\,918 = 759\,350\,217\,756$; cộng $2^{35}$ rồi chia $2^{36}$ được 11,55; lấy phần nguyên: 11.

$$Y = \begin{bmatrix} 11 & 0 & 11 & 0 \\ 15 & 0 & 1 & 0 \end{bmatrix}$$ (INT8, ghi ra DRAM tại BASE_ADDR_OUT, 1 byte mỗi phần tử)

# 7 Toàn bộ thuật toán

Ghép các mục trên lại: phần mềm chuẩn bị số nguyên trên PC, phần cứng chạy 6 pha, kết quả là Y (hoặc C) trong DRAM. Đoạn dưới đây là thuật toán đầy đủ, theo đúng thứ tự phần cứng làm.

## 7.1 Chuẩn bị trên PC (một lần cho mỗi mô hình)

1. Huấn luyện mô hình số thực; chọn $s_x, s_w, s_y$ cho từng lớp.
2. Lượng tử hóa: $q_w = \text{round}(W/s_w) \to$ B (INT8); $b_q = \text{round}(b/(s_x s_w)) \to$ bias (INT32); $M = s_x s_w / s_y \to (M_0, sh)$.
3. Đệm 0 để N, K là bội của 4; xếp row-major, little-endian; nạp vào DRAM.

## 7.2 Trên accelerator (mỗi lần gọi)

```
CHECK_CFG : kiem tra M, N, K, dia chi, tham so
LOAD_BIAS : bias[0..N-1]  <- DRAM          (neu POST_EN)
LOAD_A    : A[0..M-1][0..K-1] <- DRAM
LOAD_B    : B[0..K-1][0..N-1] <- DRAM
COMPUTE   : moi tile (i0, j0):
              moi k:  PE(r,c): acc += A[i0+r][k] * B[k][j0+c]   (co skew)
              k = K-1: res <- acc
              moi hang r hop le (moi chu ky mot hang):
                v = res[r][0..P-1]                    # INT32
                neu POST_EN:
                  acc2 = v + bias[j0..j0+P-1]
                  y    = (acc2 * M0 + 2^(30+sh)) >> (31+sh)
                  neu RELU_EN: y = max(y, 0)
                  v    = clamp(y, -128, 127)          # INT8
                ghi v vao buffer C (mask cot j0+c < N)
STORE_C   : buffer C -> DRAM  (M*N word INT32, hoac M*N/4 word INT8)
DONE
```

## 7.3 Khi lớp lớn hơn 32

**Bảng 7.1:** Chia lớp lớn thành nhiều lần gọi

| Chia theo | Cách làm | Lưu ý về bias và VPU |
|---|---|---|
| M | Mỗi lần gọi lấy 32 hàng của A, ghi 32 hàng của kết quả | Dùng VPU bình thường: mỗi hàng độc lập |
| N | Mỗi lần gọi lấy một dải cột của B (đã xếp sẵn), ra một dải cột của kết quả | Dùng VPU bình thường, với đoạn bias tương ứng của dải cột |
| K | Mỗi lần gọi tính một tổng riêng phần $C_t$ trên một khúc K; kết quả thật là $\sum_t C_t$ | **Phải** đặt `POST_EN` = 0 cho mọi khúc. CPU cộng các $C_t$, rồi tự làm bước 1–5. Nếu bật VPU từng khúc, bias bị cộng nhiều lần và requantize bị làm trên tổng chưa đủ |

**LỚP CUỐI**
Lớp cuối của mạng phân loại thường chỉ cần argmax (cột có giá trị lớn nhất). Có thể đặt `POST_EN` = 0 để giữ C ở INT32 (CPU tự cộng bias) và lấy argmax trên giá trị chính xác hơn; hoặc bật VPU với `RELU_EN` = 0 nếu chấp nhận độ phân giải INT8.

# 8 Tinh chỉnh yêu cầu

Đối chiếu với lý thuyết ở trên, yêu cầu VPU trong spec v1.0 (FUN-03) đã **đúng chuẩn**: bias INT32 cùng scale với C, cộng trước requantize, requantize bằng $M_0$ và dịch, rồi ReLU và kẹp. Bảng dưới là các điểm nên bổ sung hoặc cần bạn quyết định.

**Bảng 8.1:** Các điểm tinh chỉnh

| # | Điểm | Hiện trạng spec v1.0 | Đề xuất |
|---|---|---|---|
| 1 | Bias INT32, scale $s_x s_w$, cộng trước requantize | Đúng (FUN-03, FUN-04) | Giữ nguyên |
| 2 | Ràng buộc $M < 1$ | Ngầm định do $sh \geq 0$, chưa ghi rõ | Ghi thêm vào FUN-04: phần mềm phải bảo đảm $M_0 \cdot 2^{-(31+sh)} < 1$. Thực tế $s_y$ luôn đủ lớn để điều này đúng |
| 3 | Lớp chia K | Không nói gì | Không cần sửa spec; ghi vào hướng dẫn firmware: khúc K luôn dùng `POST_EN` = 0 (bảng 7.1) |
| 4 | Per-tensor hay per-channel | Một cặp ($M_0$, $sh$) cho cả lớp | Giữ per-tensor: đủ chính xác cho MLP và CNN nhỏ. Per-channel (mỗi cột j một cặp) tốt hơn một chút nhưng cần thêm 2 vector tham số và thanh ghi địa chỉ, tức phải đổi register map đã khóa. **Cần bạn quyết định ngay** nếu muốn để ngỏ khả năng này |
| 5 | Zero-point | Đối xứng, không zero-point | Giữ. Hệ quả: sau ReLU, Y chỉ dùng 0 … 127, bỏ phí 1 bit; ảnh hưởng nhỏ với workload của đồ án |
| 6 | Kiểu làm tròn | Nửa lên (FUN-03) | Giữ. Lưu ý: khác TFLite (làm tròn nửa ra xa 0) ở các số âm đúng nửa; golden model phải dùng đúng công thức của spec, không lấy kết quả TFLite để so từng bit |
| 7 | Đầu vào lớp đầu | A phải là INT8 đối xứng | Ảnh digits có pixel 0 … 16 nên dùng thẳng $q_x$ = pixel, $s_x$ = 1/16 hoặc tương tự |